Word2Vec

MSIS 521

Professor Hossein Ghasemkhani

FOSTER SCHOOL OF BUSINESS
UNIVERSITY OF WASHINGTON

1

Word Embedding

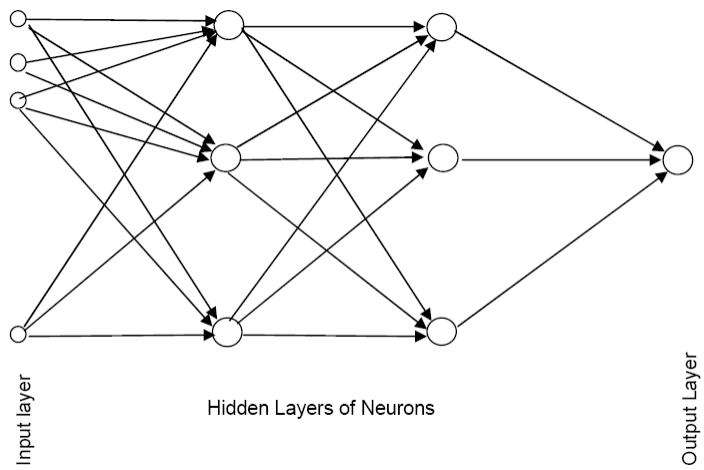
- Vector representation of a word
  - Representation of document vocabulary
  - Frequency-based vs. prediction-based
- Word2Vec
  - Prediction-based
  - Technique to learn word embeddings using a shallow neural network
  - Tomas Mikolov in 2013 at Google
- Capture the context of a word in a document
  - Semantic and syntactic similarity
  - Relationship with other words

FOSTER SCHOOL OF BUSINESS

2

2

Neural Network – A Brief Introduction



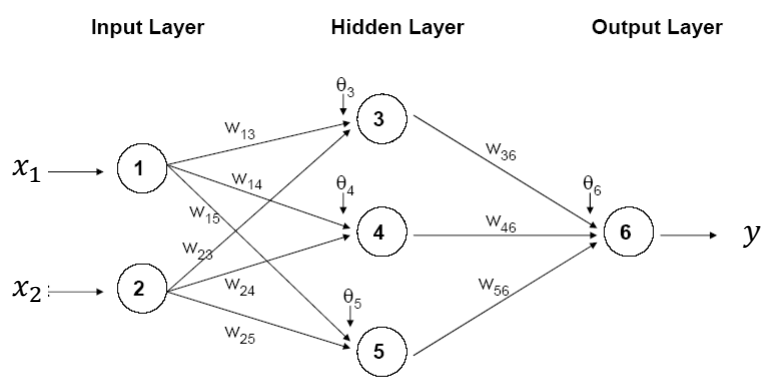
FOSTER SCHOOL OF BUSINESS

3

3

Model Parameters

- Weights: $\boldsymbol{w}$
- Biases: $\boldsymbol{\theta}$



FOSTER SCHOOL OF BUSINESS

4

4

Neurons on Hidden Layer

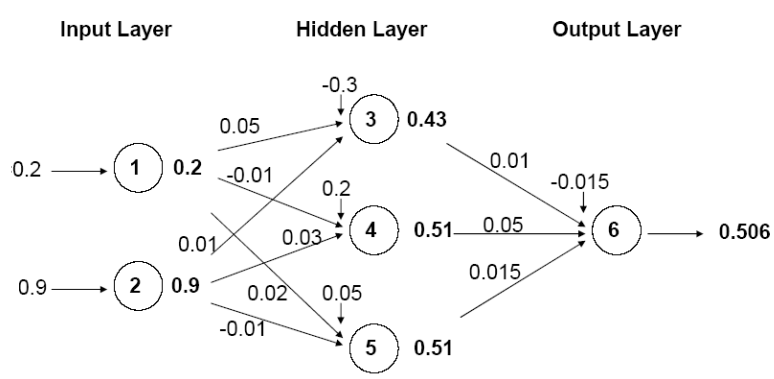
- Take inputs from the nodes on the previous layer (such as Input Layer)
- Use weights to aggregate them
- Apply the bias
- Produce an intermediate output with an activation function such as Sigmoid

$$output_j = g\left(\theta_j + \sum_{i=1}^{P} w_{ij}\, x_i\right)$$

5

5

Initial Pass of the Network



6

6

Training with Back Propagation

- Output from output node *k*: $\hat{y}_k$
- Error associated with that node, for example:

$$L = \frac{1}{2}(y - \hat{y})^2$$

- This is like ordinary error adjusted for the specific form of activation function
- Updating using gradient descent

7

7

Error to Update Parameters

- α reflects the learning rate or weight decay parameter

$$w = w - \alpha \frac{\partial L}{\partial w}$$

8

8

One-hot Vectors

- Two sentences
  - MSIS is a good program
  - MSIS is a great program
- Vocabulary: {MSIS, is, a, good, great, program}
  - Spans a 6-dimensional space
- Words as one-hot vectors
  - MSIS = [1, 0, 0, 0, 0, 0]
  - good = [0, 0, 0, 1, 0, 0]
  - great = [0, 0, 0, 0, 1, 0]
  - …
  - Words are independent of each other!

FOSTER SCHOOL OF BUSINESS

9

9

Word2Vec

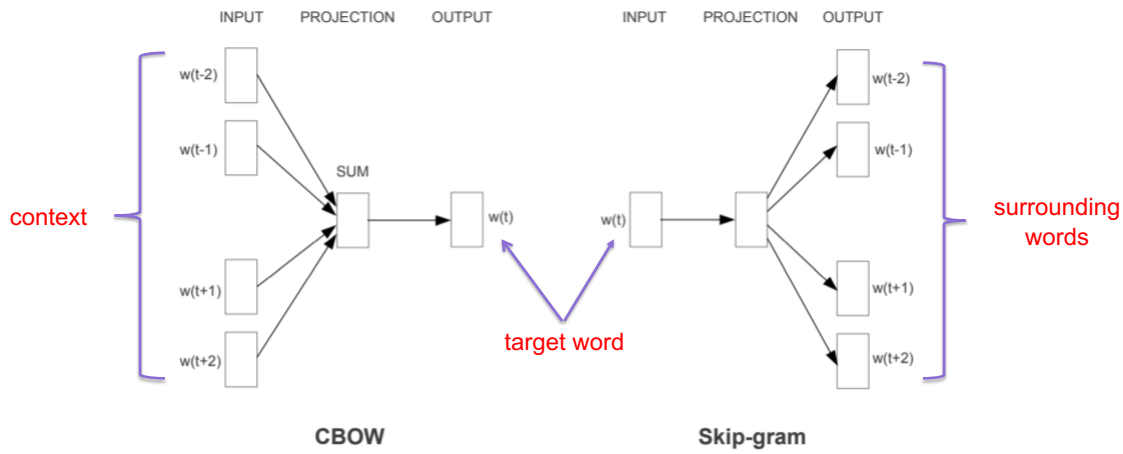
- Dependence of one word on other words
  - Similarity
    - good, great
  - Relationship
    - great program
- Two methods based on neural networks
  - Continuous Bag of Words (CBOW)
  - Skip Gram

FOSTER SCHOOL OF BUSINESS

10

10

CBOW and Skip-gram



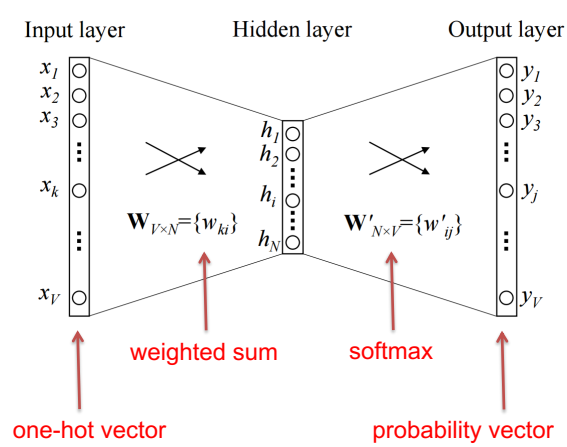
FOSTER SCHOOL OF BUSINESS

11

11

CBOW

- Use the distributed representations of context (surrounding words) to predict the target word
- Input "*great*" to predict "*program*"
- *W* are weight matrices
- Target word is represented by vector
  - $[h_1, h_2, \ldots, h_N]$



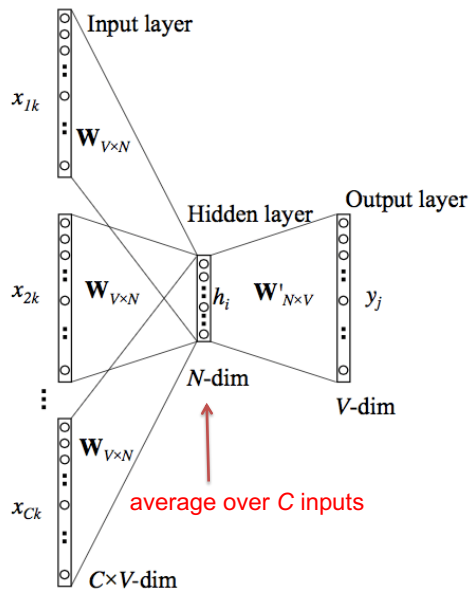
FOSTER SCHOOL OF BUSINESS

12

12

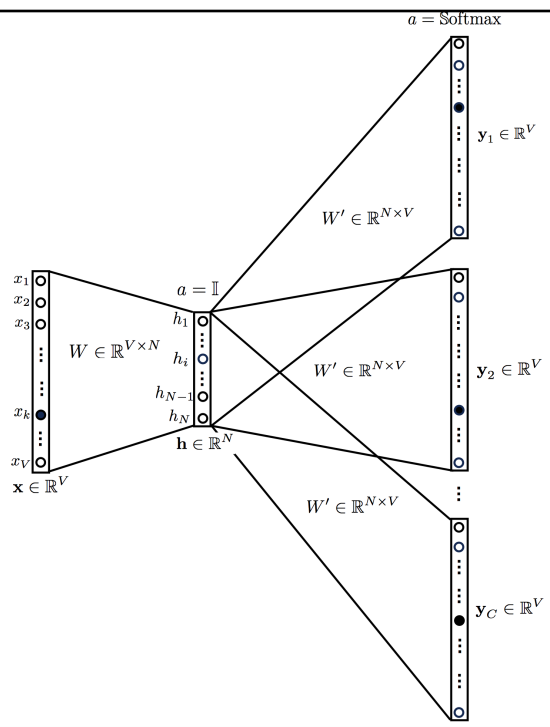
CBOW – Multiple Context Words

- Input multiple context words



Skip-gram

- Use target word to predict the context



CBOW vs. Skip-gram

- CBOW
  - Many context words to predict one target word
  - Smooth out over the distribution
  - Very good performance when input data is not so large
- Skip-gram
  - More fine-grained
  - More accurate embeddings with a large data set
  - Skip-gram with negative sub-sampling outperforms other methods generally

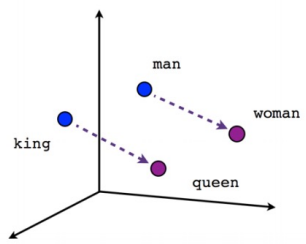
FOSTER SCHOOL OF BUSINESS

15

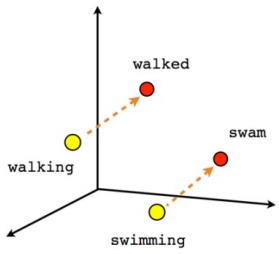
15

Performance

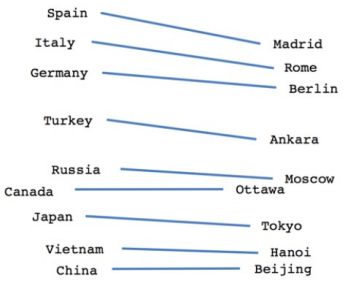
- Vectors capture useful semantic information about words and their relationships



Male-Female



Verb tense



Country-Capital

King – Man + Woman = Queen

FOSTER SCHOOL OF BUSINESS

16

16

Python Implementation

- Using genism
  - pip install gensim

- Prepare data in list of lists (of words)

```
import pandas as pd
import gensim # in console: pip install gensim

df_1 = pd.read_csv("quotetags.csv")
tag = [row.split(',') for row in df_1['Tags']]

model = gensim.models.Word2Vec(tag, min_count = 1, vector_size = 20, workers = 3, window = 3, sg = 1)

model.wv.__getitem__('Albert Einstein')
```

Number of Dimensions

Maximum distance to target word

sg:
0:CBOW
1: Skip-gram

Minimum appearance of a word

Parallel processes for training

FOSTER SCHOOL OF BUSINESS

17

17

Python Implementation

- Similarity

```
model.wv.similarity('Albert Einstein', 'Marilyn Monroe')
```

```
model.wv.most_similar('Mark Twain')[:5]
```

- t-SNE visualization

```
authors = df_1['Author'].unique().tolist()
```

```
from sklearn.manifold import TSNE
X = model.wv.__getitem__(authors)
tsne_model = TSNE(perplexity=5, n_components=2, init="pca", n_iter=1000, random_state=100)
Y = tsne_model.fit_transform(X)
```

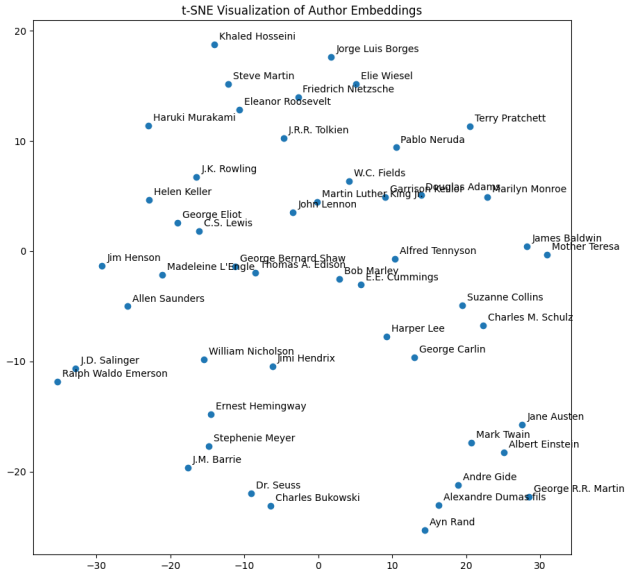
```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10,10))
ax.scatter(Y[:, 0], Y[:, 1])
for i, author in enumerate(authors):
    plt.annotate(author, xy=(Y[i, 0], Y[i, 1]))

plt.savefig("tsne-quotes.png")
plt.show()
```

FOSTER SCHOOL OF BUSINESS

18

18

Python Implementation



FOSTER SCHOOL OF BUSINESS

19

19

Principal Components Analysis (PCA)

- **Goal**: Reduce a set of numerical variables.
- **Idea**: Remove the overlap of information between these variable
  - "Information" is measured by the sum of the variances of the variables
- **Final product**: A smaller number of numerical variables that contain most of the information

FOSTER SCHOOL OF BUSINESS

20

20

Principal Components

How does PCA do this?

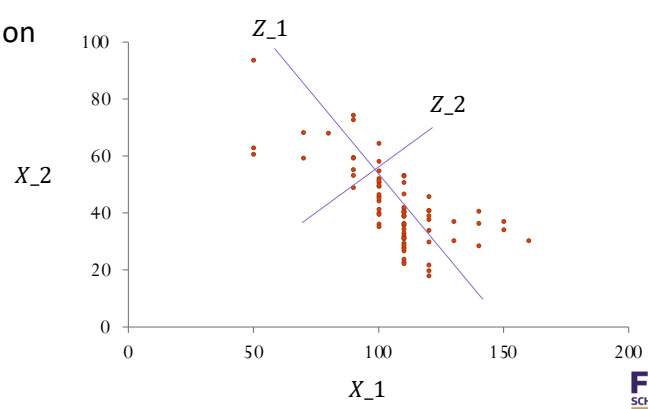
- Create new variables that are linear combinations of the original variables
  - They are weighted averages of the original variables
- These linear combinations are uncorrelated
  - no information overlap
- Only a few of them contain most of the original information
- The new variables are called *principal components*

21

21

Principal Components

- Z_1 and Z_2 are two linear combinations of original variables X_1 and X_2
  - X_1 and X_2 are (highly) correlated
  - Z_1 and Z_2 are orthogonal
- Z_1 has the highest variation (spread of values)
- Z_2 has the lowest variation



22

22